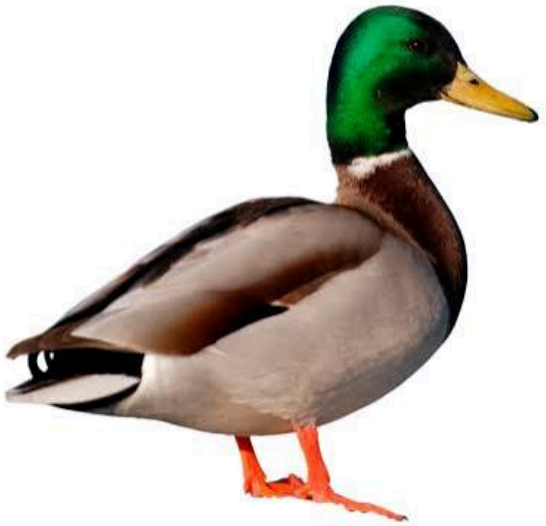
Ciência de Dados Aplicada ao Direito I

Aula 17

Funções em lote e classes



Prof. Bruno da Cunha de Oliveira - 2026/2

A função dentro do for

Insper

Chamar a função dentro do for

```
def prazo_em_dobro(dias):
    return dias * 2

prazos = [15, 5]
dobrados = []
for simples in prazos:
    dobrado = prazo_em_dobro(simples)
    dobrados.append(dobrado)
print(dobrados)
```

A parte tem prazo em dobro em dois prazos. O que aparece na tela?

A função dentro do for, passo a passo

Animação no navegador: `animacao-lote.html`, cenário `prazos`.

Uma chamada por volta

RESPOSTA

```
[30, 10]
```

A linha da chamada está escrita uma vez, e o `for` a repete. Na primeira volta, `dias` recebe 15; na segunda, 5. Aqui o `return` devolve a conta direto, sem guardar o resultado numa variável antes.

A triagem de dois casos de estelionato

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

casos = [{"investigado": "Marina Alves", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": False},
         {"investigado": "Otávio Bastos", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": True}]
```

Escrevam o `for` que aplica a função a cada caso e guarda as respostas na lista `conclusoes`.

Os campos do caso viram os argumentos

RESPOSTA

```
conclusoes = []
for caso in casos:
    conclusao = configura_estelionato(
        caso["vantagem"], caso["prejuizo"],
        caso["erro"], caso["fraude"])
    conclusoes.append(conclusao)
```

`conclusoes` fica com `[False, True]`. A função é a mesma de um caso só, e quem repete é o `for`.

Função que chama função

Insper

Dois passos, duas funções

```
def prazo_em_dobro(dias):
    return dias * 2

def dias_restantes(prazo, decorridos):
    return prazo - decorridos

total = prazo_em_dobro(15)
faltam = dias_restantes(total, 12)
print(faltam)
```

Apelação: 15 dias, em dobro, com 12 já decorridos. O que aparece na tela?

Uma função depois da outra, passo a passo

Animação no navegador: `animacao-encadeamento.html`, cenário `sequencia`.

A saída de uma é a entrada da outra

RESPOSTA

```
18
```

`prazo_em_dobro` devolveu 30, que ficou guardado em `total`. `total` entrou como argumento de `dias_restantes`, e 30 menos 12 dá 18.

Uma função para os dois passos

```
def prazo_em_dobro(dias):
    return dias * 2
```

Escrevam a função `restantes_em_dobro`. Ela recebe `dias` e `decorridos` e devolve quantos dias restam de um prazo em dobro.

Ela deve chamar `prazo_em_dobro`, sem repetir a conta do dobro.

Uma função dentro da outra, passo a passo

Animação no navegador: `animacao-encadeamento.html`, cenário `dentro`.

Uma função chama a outra

RESPOSTA

```
def restantes_em_dobro(dias, decorridos):
    prazo = prazo_em_dobro(dias)
    return prazo - decorridos

faltam = restantes_em_dobro(15, 12)
```

`restantes_em_dobro` chama `prazo_em_dobro` e espera a resposta. `faltam` guarda 18, e a regra do dobro continua escrita num lugar só.

Um resultado diferente do esperado

```python
def prazo_em_dobro(dias):
    return dias * 2

def restantes_em_dobro(dias, decorridos):
    prazo = prazo_em_dobro(dias)
    return decorridos - prazo

print(restantes_em_dobro(15, 12))
```

Eu esperava 18, e apareceu −18. Que chamada vocês fariam para descobrir em qual das duas funções está o defeito?

Conferir cada função sozinha

RESPOSTA

```
print(prazo_em_dobro(15))
```

```
30
```

Sozinha, `prazo_em_dobro` responde certo. O defeito está em `restantes_em_dobro`: a conta saiu invertida, `decorridos - prazo`. Conferir cada função sozinha diz onde procurar.

A ordem dos argumentos

Insper

Ler uma chamada com quatro argumentos

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

resposta = configura_estelionato(True, True, False, True)
```

Qual dos quatro fatos não foi apurado neste caso? Como vocês descobriram?

Argumento nomeado

RESPOSTA

Foi `erro`, o terceiro. Para saber, é preciso contar a posição e voltar à definição.

```
resposta = configura_estelionato(
    vantagem=True, prejuizo=True,
    erro=False, fraude=True)
```

Cada argumento pode ir com o nome do parâmetro. A chamada passa a dizer os fatos.

Argumentos nomeados em outra ordem

```python
def dias_restantes(prazo, decorridos):
    return prazo - decorridos

print(dias_restantes(30, 12))
print(dias_restantes(decorridos=12, prazo=30))
print(dias_restantes(12, 30))
```

O que aparece na tela?

Com nome, a ordem não importa

RESPOSTA

```
18
18
-18
```

Sem nome, vale a posição, e a terceira chamada trocou o prazo e os dias decorridos. Com nome, cada valor vai para o parâmetro que a chamada indica, em qualquer ordem.

Menos argumentos que parâmetros

Uma chamada com um argumento a menos

```
def prazo_final(dias, dobro=False):
    if dobro:
        prazo = dias * 2
    else:
        prazo = dias
    return prazo

print(prazo_final(15))
print(prazo_final(15, dobro=True))
```

A primeira chamada traz um argumento só. O que aparece na tela?

Parâmetro opcional

RESPOSTA

```
15
30
```

`dobro=False` na definição dá ao parâmetro um valor padrão. Sem o segundo argumento, `dobro` fica com `False`. Quase toda parte conta prazo simples, então a chamada mais comum fica mais curta, e só quem tem prazo em dobro escreve `dobro=True`.

Uma função que preenche um modelo

```
modelo = "Manifeste-se a parte em <DIAS> dias."

def preencher(modelo, dias):
    modelo.replace("<DIAS>", str(dias))
    return modelo

print(preencher(modelo, 30))
```

O que aparece na tela?

Guardar o que o replace devolve

RESPOSTA

```
Manifeste-se a parte em <DIAS> dias.
```

`.replace()` devolve um texto novo e não altera `modelo`. A função jogou fora o texto preenchido e devolveu o modelo como chegou.

```
    texto = modelo.replace("<DIAS>", str(dias))
    return texto
```

No PrairieLearn: exercício 6

Insper

No PrairieLearn: Desafio V

Insper

Um programa com o juscraper

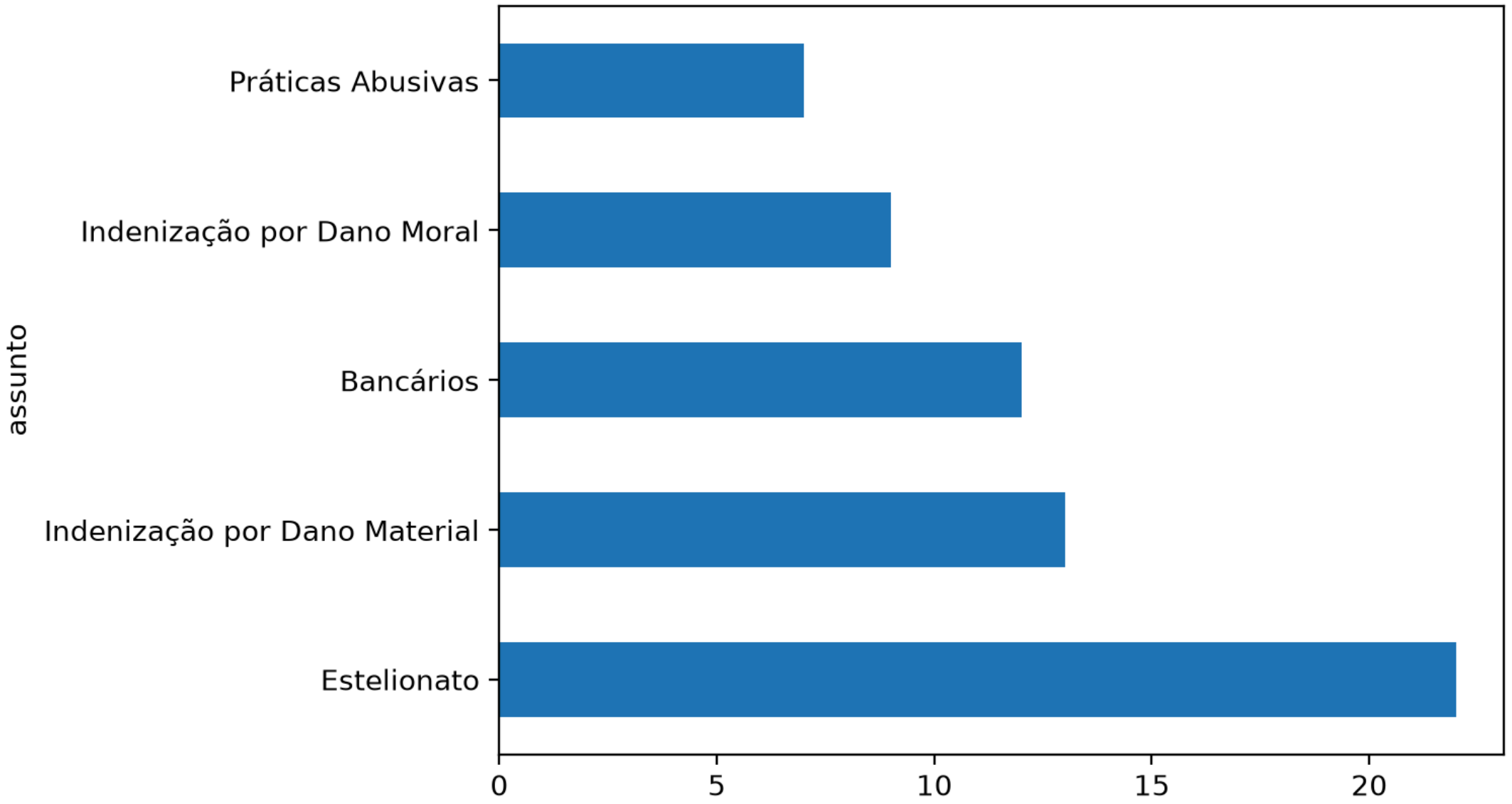
Leiam sozinhos. Que partes vocês já entendem, e que partes ainda são mistério?

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

O gráfico que esse programa produz




SONY
Discman
GAME BOY COLOR
TYCO
TYCO
27MHz
Oct 28, Oct 28
10:09

Python não vem com pilha

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

NÃO VEM

Tabelas. Gráficos. Leitura de PDF. Coleta de decisões em tribunais. Modelos de linguagem.

É quase tudo o que um trabalho com dados precisa.

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

NÃO VEM

Tabelas. Gráficos. Leitura de PDF. Coleta de decisões em tribunais. Modelos de linguagem.

É quase tudo o que um trabalho com dados precisa.

O que não vem se instala: são as bibliotecas, código que outra pessoa escreveu e publicou. O juscraper é uma delas.

O ponto depois do valor

Insper

O ponto depois de um texto e de uma lista

```
tribunal = "TJSP"
pecas = ["petição inicial", "contestação"]

print(tribunal.replace("SP", "RJ"))
pecas.append("sentença")
print(pecas)
print(pecas.replace("sentença", "acórdão"))
```

O que aparece na tela?

Cada tipo tem os seus métodos

RESPOSTA

```
TJRJ
['petição inicial', 'contestação', 'sentença']
AttributeError: 'list' object has no attribute 'replace'
```

`.replace()` é método de texto, e `.append()`, de lista. O ponto pede a ação ao valor que vem antes dele, e uma lista não sabe fazer `.replace()`.

Função ou método

Elemento	Como se usa	Exemplos
função	vive solta, e o valor entra entre os parênteses	`len(pecas)`, `print(pecas)`
método	mora dentro de um tipo, e age sobre o valor antes do ponto	`pecas.append("sentença")`, `tribunal.replace("SP", "RJ")`

Um método é uma função que já sabe sobre qual valor vai agir.

Criar um tipo

Insper

Do dicionário ao objeto

DICIONÁRIO E FUNÇÃO SOLTA

```
processo = {"classe": "Cível", "valor": 25000}
alto_valor(processo)
```

OBJETO E MÉTODO

```
processo = Processo("Cível", 25000)
processo.alto_valor()
```

No dicionário, os dados ficam num lugar e a pergunta, em outro. No objeto, os dados e o que se pergunta a eles andam juntos.

Criar um objeto, passo a passo

Animação no navegador: `animacao-objetos.html`, cenário `criar`.

class, `__init__` e self

```
class Processo:
    def __init__(self, classe, valor):
        self.classe = classe
        self.valor = valor

    def alto_valor(self):
        return self.valor > 10000
```

`class` cria o molde. `__init__` roda quando um objeto nasce e guarda os dados dele. `self` é o próprio objeto: `self.valor` é o valor deste processo.

Atributo e método

```
caso = Processo("Cível", 25000)
```

Elemento	O que é	Neste caso
atributo	um dado que o objeto guarda; sem parênteses	`caso.valor`
método	uma ação que o objeto executa; com parênteses	`caso.alto_valor()`

Escrever a classe Processo

Escrevam a classe `Processo`, com dois atributos e um método.

Os atributos são `numero`, o número do processo, e `valor_sm`, o valor da causa em salários mínimos.

O método `dentro_do_teto()` devolve `True` quando o valor da causa não passa de 40 salários mínimos, o teto de valor do Juizado Especial Cível, e `False` quando passa.

A classe Processo

RESPOSTA

```
class Processo:
    def __init__(self, numero, valor_sm):
        self.numero = numero
        self.valor_sm = valor_sm

    def dentro_do_teto(self):
        return self.valor_sm <= 40
```

A comparação já vale `True` ou `False`, e o `return` a entrega.

Uma lista de objetos, passo a passo

Animação no navegador: `animacao-objetos.html`, cenário `lista`.

Uma lista de processos

Com a classe `Processo` que vocês escreveram, criem dois processos e guardem os dois numa lista:

Número	Valor da causa
`0000123-27.2024.8.99.0001`	15 salários mínimos
`0000456-76.2024.8.99.0001`	50 salários mínimos

Depois, percorram a lista e mostrem na tela o número dos processos que estão dentro do teto.

Percorrer a lista de objetos

RESPOSTA

```
processos = [
    Processo("0000123-27.2024.8.99.0001", 15),
    Processo("0000456-76.2024.8.99.0001", 50),
]
for processo in processos:
    if processo.dentro_do_teto():
        print(processo.numero)
```

Só o primeiro número aparece: 50 salários mínimos passam do teto.

Um método sem self

```
class Processo:
    def __init__(self, numero, valor_sm):
        self.numero = numero
        self.valor_sm = valor_sm

    def dentro_do_teto():
        return self.valor_sm <= 40
causa = Processo("0000123-27.2024.8.99.0001", 15)
print(causa.dentro_do_teto())
```

Tirei o `self` do método. O que aparece na tela?

O objeto antes do ponto é o self

RESPOSTA

```
TypeError: Processo.dentro_do_teto() takes 0 positional
arguments but 1 was given
```

Ninguém escreveu argumento entre os parênteses. Quem passou foi o Python: em `causa.dentro_do_teto()`, ele entrega o objeto antes do ponto como primeiro argumento do método, e o método não tinha o parâmetro `self` para recebê-lo.